

# FeedBack Project

A comprehensive, role-based academic feedback system built with **Flask**, **SQLAlchemy**, and **MySQL**. This application facilitates anonymous student feedback collection, teacher evaluation, and administrative reporting across four distinct roles: **Admin**, **HOD**, **Teacher**, and **Student**.

## Features

### Multi-Role Authentication

- Dedicated login portals for **Admins/Teachers/Students** and a separate **Management Portal** for HODs.
- Session-based authentication with **role-specific route protection** — users cannot access pages outside their role.
- **Login audit logging** — every login attempt (success or failure) is recorded in `logs/login_audit.log` with timestamp, role, IP address, and browser.

### Default Password Warning

- On every login, the system automatically **detects if the user is still using their default password**.
- If detected, a **red warning popup** appears in the bottom-right corner of the screen and stays visible for **5 seconds**, reminding the user to change their password.
- The popup appears **on every login session** until the password is changed — it will not reappear after a successful password update.

Default passwords by role:

| Role    | Default Password             | Where to Change                |
|---------|------------------------------|--------------------------------|
| HOD     | HOD@123                      | Management Portal → Security   |
| Admin   | Admin@123                    | Admin Portal → Change Password |
| Teacher | {EmpID}@123 (e.g. T101@123 ) | Teacher Portal → Security      |
| Student | Pass@123                     | Student Portal → Security      |

### Anonymous Feedback System

- **Token-based submissions** ensure complete student anonymity — feedback cannot be traced back to individuals.
- Dynamically generated forms based on student class, division, batch, and elective subjects.
- Separate configurable question sets for **Theory** and **Practical** subjects (stored as JSON files).

- Each student can only submit feedback **once per session** — subsequent logins show the submitted status.

## Admin Panel

- Manage **Teachers**, **Subjects**, **Sessions**, and **Allocations** (teacher → subject → class mappings).
- **Bulk CSV upload** for importing students, teachers, and subjects — supports upsert (add or update).
- Assign **Class Teachers** to specific semester/division combinations.
- **Promote / Rollback students** across semesters with password-confirmed destructive actions.
- **Semester-wise elective configuration** — set the number of electives per semester.
- View detailed feedback **reports** with per-question score breakdowns, response counts, and subjective comments.
- Filter reports by teacher, subject, or semester.
- **Dashboard** with live counts of total teachers and students.

## HOD (Head of Department) Portal

- Create and manage **Admin** accounts (with default password `Admin@123` ).
- Review and **approve/reject** feedback reports submitted by teachers individually or in bulk.
- Access consolidated feedback results across all sessions and teachers.
- **Analyze results** with a dedicated analysis view separate from the approval workflow.
- **Dashboard** with live counts of total teachers and students.

## Teacher Portal

- View personal feedback scores, per-class breakdowns, and student subjective comments.
- Manage student records: add, edit, and delete students; assign elective subjects with division/batch.
- **PRN field is read-only** during student edits to prevent accidental identity changes.
- **Acknowledge** feedback reports for HOD review, with per-subject and bulk-agree options.
- Agreement is only allowed **after a session is terminated** by the Admin.

## Student Portal

- Generate a one-time anonymous **feedback token** per active session.
- Submit feedback for all allocated subjects in a single, guided form.
- View past feedback submission **history** (completed vs. missed sessions).
- Change password from the **Security** section.

## Automated Backups

- **Daily SQL backups** via `mysqldump` with automatic scheduling on production server startup.
- Backups stored in the `backups/` directory with date-stamped filenames ( `backup_YYYY-MM-DD.sql` ).
- **7-day rolling retention** — older backups are automatically purged.
- Can also be triggered manually at any time: `python backup.py` .

## Login Audit Log

- Every login attempt is written to `logs/login_audit.log`.
- Each entry records: **Timestamp** | **Status** | **Role** | **User ID** | **IP Address** | **Browser**.
- Supports reverse-proxy environments via `X-Forwarded-For` header.

## Prerequisites

| Requirement  | Version                     |
|--------------|-----------------------------|
| Python       | 3.8+                        |
| MySQL Server | 5.7+ (XAMPP/WAMP or native) |
| pip          | Latest recommended          |

## Installation & Setup

### 1. Clone the Repository

```
git clone <your-repository-url>
cd FeedBack_Project
```

### 2. Create a Virtual Environment (Recommended)

```
python -m venv venv

# On Windows:
venv\Scripts\activate

# On Mac/Linux:
source venv/bin/activate
```

### 3. Install Dependencies

```
pip install -r requirements.txt
```

### 4. Configure Environment Variables

Create a `.env` file in the project root:

```
DB_HOST=localhost
DB_USERNAME=root
DB_PASSWORD=your_mysql_password
DB_NAME=feedback_db
SECRET_KEY=generate_a_strong_random_secret_key
```

[!NOTE]

If your local MySQL doesn't use a password, leave `DB_PASSWORD=` blank.

The app will **automatically create the database and tables** on first startup.

## Usage

### Development Server

For testing and development with hot-reload and detailed error pages:

```
python run.py
```

Access at: `http://127.0.0.1:5000`

### Production Server

For deployment or allowing concurrent access over your local network:

```
python serve.py
```

Access at: `http://0.0.0.0:80` (falls back to port `5000` if port `80` is unavailable or requires admin privileges).

[!TIP]

Share your machine's IP address (e.g., `http://192.168.x.x`) with students so they can access the feedback portal from their own devices.

The production server ( `serve.py` ) also:

- Uses **Waitress** WSGI server (32 threads) for concurrent access.
- Logs all HTTP requests via **Paste TransLogger**.
- **Runs a database backup on startup**.
- **Schedules daily automatic backups** (every 24 hours) in a background thread.

## Manual Backup

To trigger a database backup manually at any time:

```
python backup.py
```

Backups are saved to the `backups/` directory. Only the last 7 days of backups are kept.

| Server      | Command                      | Best For                            |
|-------------|------------------------------|-------------------------------------|
| Development | <code>python run.py</code>   | Coding, debugging, hot-reload       |
| Production  | <code>python serve.py</code> | Real usage, network access, backups |

## Getting Started — First-Time Setup

## 1. Root HOD Account (Auto-Created)

The root HOD account is **automatically created** the first time you start the server ( `python run.py` or `python serve.py` ). No manual setup is needed.

### Default HOD credentials:

| Field    | Value   |
|----------|---------|
| ID       | HOD101  |
| Password | HOD@123 |

[!IMPORTANT]

Change the default HOD password immediately after your first login. A **red warning popup** will appear on every login until the password is changed. To customize the initial credentials, edit `create_hod.py` before the first server start.

## 2. Setup Hierarchy

```

graph TD
    HOD -- creates --> Admins
    Admins -- creates --> TeachersSubjects[Teachers & Subjects]
    TeachersSubjects -- allocate to classes --> Students
    Students -- submit anonymous feedback --> Feedback[Feedback]

```

### Step-by-step workflow:

1. **HOD** logs in at `/management_login` and creates one or more **Admin** accounts.
2. **Admin** logs in at `/login` , adds **Teachers** and **Subjects**, and creates **Allocations** (which teacher teaches which subject to which class).
3. **Admin** assigns a **Class Teacher** for each semester/division — this teacher manages student records.
4. **Class Teacher** adds students individually or via CSV bulk upload, assigning their batch and elective subjects.
5. **Admin** opens a **Session** (makes it LIVE) to allow students to submit feedback.
6. **Students** log in, generate a token, and submit their feedback form.
7. **Admin** terminates the session when feedback collection is complete.
8. **Teachers** log in to view their results and **agree** to the feedback reports.
9. **HOD** reviews and **approves** the reports.

### 3. Important Routes

| Route                           | Portal            | Who Can Access             |
|---------------------------------|-------------------|----------------------------|
| <code>/login</code>             | Main Portal       | Admins, Teachers, Students |
| <code>/management_login</code>  | Management Portal | HODs only                  |
| <code>/admin/dashboard</code>   | Admin Panel       | Admins                     |
| <code>/teacher/dashboard</code> | Teacher Portal    | Teachers                   |
| <code>/hod/dashboard</code>     | HOD Portal        | HODs                       |

### Project Structure



## Database Schema

The application uses **14 interconnected models**:

| Model                  | Purpose                                                                 |
|------------------------|-------------------------------------------------------------------------|
| User                   | All users (admin, teacher, student, HOD) with role-based access         |
| Session                | Feedback collection sessions (inactive / live / terminated)             |
| Subject                | Theory/Practical subjects with elective & linked-subject (twin) support |
| StudentElective        | Maps students to their chosen elective subjects with div/batch          |
| Allocation             | Maps teacher → subject → target class (semester/division/batch)         |
| TokenLog               | Tracks whether a student has generated/submitted a token per session    |
| ValidToken             | Pool of valid anonymous tokens                                          |
| ActiveTokenMap         | Links students to their currently active token (session-scoped)         |
| FeedbackResult         | Individual question ratings per allocation                              |
| FeedbackComment        | Subjective text comments per allocation                                 |
| ClassTeacherAllocation | Assigns teachers as class teachers for specific divisions               |
| ReportApproval         | Tracks teacher acknowledgment and HOD approval of feedback reports      |
| SemesterConfig         | Configures the number of elective subjects per semester                 |

## Tech Stack

| Layer       | Technology                                                         |
|-------------|--------------------------------------------------------------------|
| Backend     | Flask, Flask-SQLAlchemy, Flask-WTF                                 |
| Database    | MySQL (via PyMySQL)                                                |
| Auth        | Werkzeug (PBKDF2-SHA256), Flask sessions                           |
| WSGI Server | Waitress (production), Flask dev server (development)              |
| Logging     | Paste TransLogger (HTTP access logs), Python logging (login audit) |



|            |                                                   |
|------------|---------------------------------------------------|
| Backups    | mysqldump (automated via <code>backup.py</code> ) |
| Templating | Jinja2                                            |

## Security Notes

[!CAUTION]

**Never commit `.env` to version control.** It contains your database credentials and secret key. The `.gitignore` already excludes it.

- **Default Password Warning:** Every user is shown a **red popup notification on login** if they are still using their default password. The warning persists on every login until the password is changed.
- **Default Passwords by Role:**
  - HOD → `HOD@123` | Admin → `Admin@123` | Teacher → `{EmpID}@123` | Student → `Pass@123`
- **Destructive Actions:** Deleting subjects or teachers **cascades** to delete all associated feedback data. These actions require **Admin password confirmation**.
- **CSRF Protection:** All form submissions and AJAX requests are protected by CSRF tokens via `Flask-WTF` .
- **Read-Only Identifiers:** PRN (students) and EmpID (teachers) fields are locked during edits to preserve data integrity.
- **Password Hashing:** All passwords are hashed with **PBKDF2-SHA256** before being stored — plaintext passwords are never saved.
- **Role Locking:** Every route verifies the user's session role. Accessing a route outside your role redirects to the correct login page.

## License

This project is for academic/educational use.